COMPTE RENDU DE TP

TP Laravel 2026-27

MCB Bank : banque en ligne avec Blade templating

Réalisation pas à pas avec le Query Builder (Laravel 10, WAMP)

Étudiant : Fosso Ulrich Dalkin

1. Présentation du TP

Cette partie présente le sujet, les trois espaces du site et l'environnement utilisé.

1.1 Objectif et périmètre fonctionnel

L'objectif du TP est de réaliser le site d'une banque en ligne, **MCB Bank**, à partir des templates HTML fournis par le formateur, en appliquant le **Blade templating** de Laravel. Le site comporte trois espaces :

Espace	Pages du template	Fonctionnalités réalisées
Connexion	login.html	Connexion du compte (3 sessions : User, Manager, Cashier)
Admin (manager) `/admin/home`	mindex, mshow, maddnew, maccounts, mfeedback, mnotice	Création de compte bancaire, détails du client, suppression du compte client, CRUD caissier, suppression de feedback, envoi de notice
Client `/client/home`	index, accounts, transfer, transfe, statements, feedback, notice	Détails du compte bancaire, transfert de fonds, relevé bancaire, note de feedback
Caissier `/cashier/home`	cindex, cindex1	Dépôt et retrait

Méthode utilisée : le Query Builder (`DB::table(...)`) vu en cours. Chaque fonctionnalité suit le même schéma : la route dans `web.php`, la méthode dans le contrôleur, la vue Blade, puis le test dans le navigateur.

1.2 Environnement de travail

Élément	Version / valeur
Serveur local	WAMP (Apache + MySQL 8.3)
PHP	8.2
Framework	Laravel 10.50
Éditeur	Visual Studio Code
Serveur de développement	`php artisan serve` → http://127.0.0.1:8000
Base de données	`mcbbank`

```
PS C:\Users\Utilisateur 1> cd C:\wamp64\www
PS C:\wamp64\www> composer create-project laravel/laravel:^10.0 MCBBank
Creating a "laravel/laravel:10.0" project at "./MCBBank"
https://repo.packagist.org could not be fully loaded (curl error 28 while downloading https://repo.packagist.org/packages.json: SSL connection timeout), package information was loaded from the local cache and may be out of date
Installing laravel/laravel (v10.0.0)
  - Installing laravel/laravel (v10.0.0): Extracting archive
Created project in C:\wamp64\www\MCBBank
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
Updating dependencies
Lock file operations: 110 installs, 0 updates, 0 removals
  - Locking brick/math (0.12.3)
  - Locking carbonphp/carbon-doctrine-types (2.1.1)
  - Locking dflydev/dot-access-data (v3.0.3)
```

Figure 1 : *Création du projet : composer create-project laravel/laravel:^10.0 MCBBank*



Figure 2 : *Lancement du serveur de développement : php artisan serve*

2. Mise en place du projet

Intégration du template, configuration de la base de données, création des tables et du premier compte.

2.1 Intégration du template dans public

Les dossiers `css`, `js`, `images` et `font-awesome` du template ont été copiés dans `public/frontend`. Les pages `.html` ont été renommées en `.blade.php` et rangées dans `resources/views/Admin` et `resources/views/Client`.

public/frontend

```
public/
└── frontend/
    ├── css/
    ├── font-awesome/
    ├── images/
    └── js/
```

Dans toutes les vues, les liens vers le CSS, le JS et les images passent par la fonction `asset()`, pour qu'ils fonctionnent quelle que soit l'adresse de la page (par exemple `/admin/home`) :

```
<link rel="stylesheet" type="text/css" href="{{ asset('frontend/css/bootstrap.min.css') }}">
<img src="{{ asset('frontend/images/logo.png') }}" width="30" height="30" alt="">
```



Figure 3 : *Explorateur de VS Code : dossier public/frontend et vues renommées en .blade.php*

2.2 Base de données et fichier .env

Une base de données `mcbbank` a été créée dans phpMyAdmin (http://localhost/phpmyadmin), puis déclarée dans le fichier `.env` :

.env

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=mcbbank
DB_USERNAME=root
DB_PASSWORD=
```

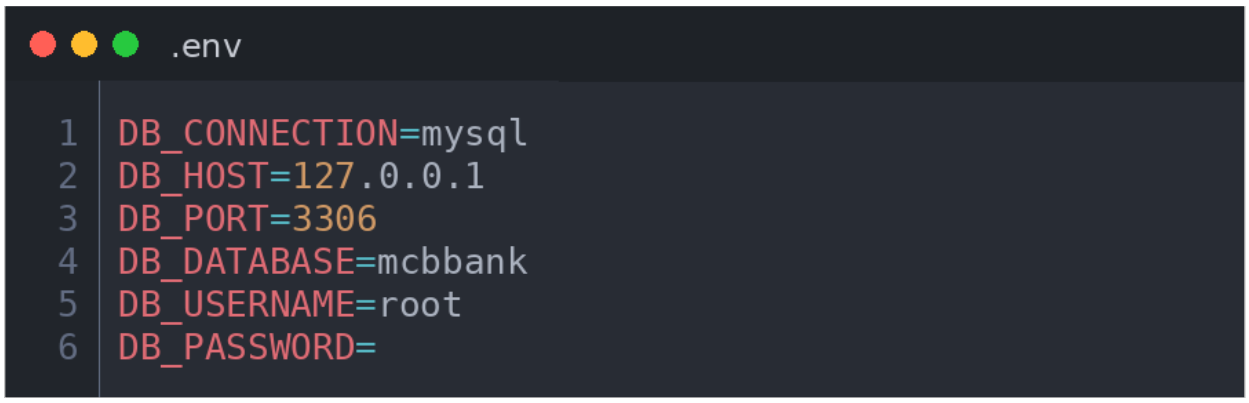


Figure 4 : *Fichier .env : lignes DB_*

Problème rencontré : « La clé est trop longue »

Le premier lancement de `php artisan migrate` a échoué avec l'erreur suivante :

```
SQLSTATE[42000]: Syntax error or access violation: 1071 La clé est trop longue.
Longueur maximale: 1000 (SQL: alter table `users` add unique `users_email_unique`(`email`))
```

Explication : sous WAMP, MySQL crée les tables avec le moteur **MyISAM**, qui limite un index à 1000 octets. Or une colonne `email` de 255 caractères en `utf8mb4` occupe jusqu'à 255 × 4 = 1020 octets. MyISAM ne gère pas non plus les clés étrangères, dont on a besoin pour `cascadeOnDelete()`.

Solution : forcer le moteur **InnoDB** dans `config/database.php`, connexion `mysql` :

`config/database.php`

```
            'strict' => true,
            // InnoDB : nécessaire sous WAMP (sinon MyISAM => "clé trop longue" et pas de clés étrangères)
            'engine' => 'InnoDB',
```

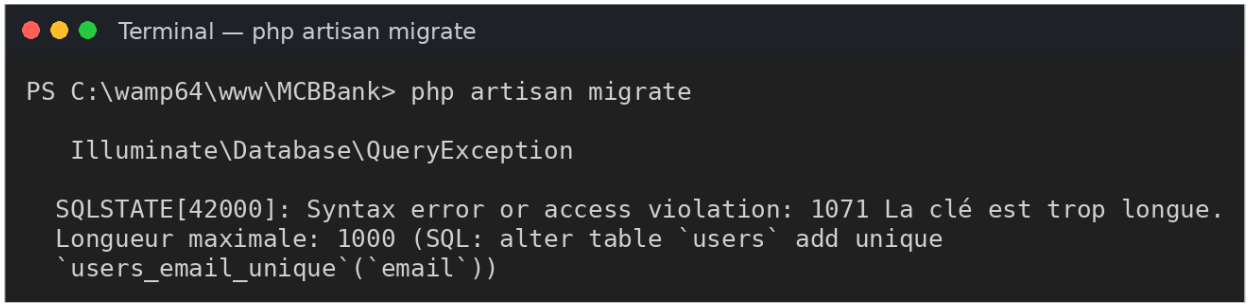


Figure 5 : *Message d'erreur « La clé est trop longue » dans le terminal*

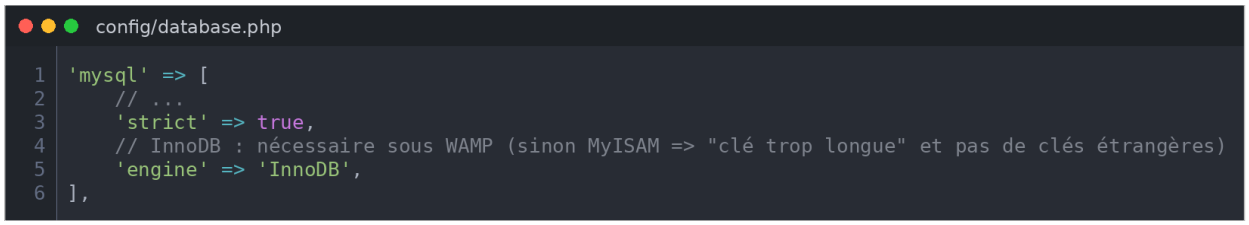


Figure 6 : *config/database.php : ligne 'engine' => 'InnoDB'*

2.3 Migrations : création des tables

Cinq tables sont nécessaires :

Table	Contenu
`staff`	le manager et les caissiers (email, password, type)

Table	Contenu
`accounts`	les comptes clients (les champs du formulaire maddnew)
`transactions`	les opérations : transfert, dépôt, retrait (pour le relevé)
`feedbacks`	les messages des clients
`notices`	les notices envoyées par la banque aux clients

Les migrations ont été créées dans cet ordre, car `accounts` doit exister avant les tables qui y font référence :

```
php artisan make:migration create_staff_table
php artisan make:migration create_accounts_table
php artisan make:migration create_transactions_table
php artisan make:migration create_feedbacks_table
php artisan make:migration create_notices_table
```

2026_10_23_122317_create_staff_table.php

```
Schema::create('staff', function (Blueprint $table) {
    $table->id();
    $table->string('email')->unique();
    $table->string('password');
    $table->string('type');             // manager ou cashier
    $table->timestamps();
});
```

2026_10_23_122318_create_accounts_table.php

```
Schema::create('accounts', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('cnic');
    $table->string('accountNo')->unique();
    $table->string('accountType');       // current ou saving
    $table->string('city');
    $table->string('address');
    $table->string('email')->unique();
    $table->string('password');
    $table->decimal('balance', 12, 2)->default(0);
    $table->string('source');            // source de revenus
    $table->string('number');            // téléphone
    $table->string('image')->nullable(); // photo
    $table->timestamps();
});
```

2026_10_23_122318_create_transactions_table.php

```
Schema::create('transactions', function (Blueprint $table) {
    $table->id();
    $table->foreignId('account_id')->constrained()->cascadeOnDelete();
    $table->string('action');            // transfer, deposit, withdraw
    $table->decimal('debit', 12, 2)->nullable();
    $table->decimal('credit', 12, 2)->nullable();
    $table->string('other')->nullable(); // n° de compte de l'autre partie
    $table->timestamps();
});
```

2026_10_23_122319_create_feedbacks_table.php

```
Schema::create('feedbacks', function (Blueprint $table) {
    $table->id();
    $table->foreignId('account_id')->constrained()->cascadeOnDelete();
    $table->text('message');
    $table->timestamps();
});
```

2026_10_23_122319_create_notices_table.php

```
Schema::create('notices', function (Blueprint $table) {
    $table->id();
    $table->foreignId('account_id')->constrained()->cascadeOnDelete();
    $table->text('notice');
    $table->timestamps();
});
```

> **À retenir :** `foreignId('account_id')->constrained()->cascadeOnDelete()` : quand on supprime un compte client, ses transactions, feedbacks et notices sont supprimés automatiquement.

Lancement des migrations :

```
php artisan migrate
```

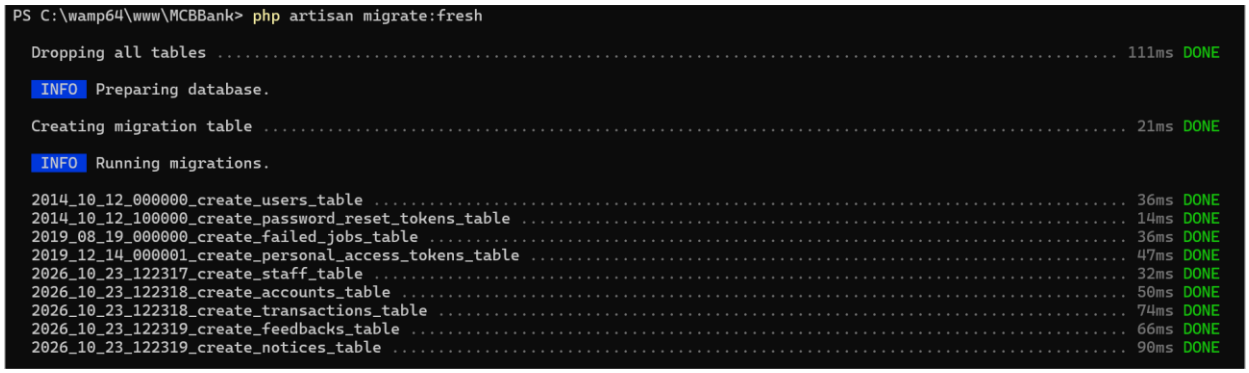


Figure 7 : *Terminal : php artisan migrate:fresh, toutes les migrations en DONE*

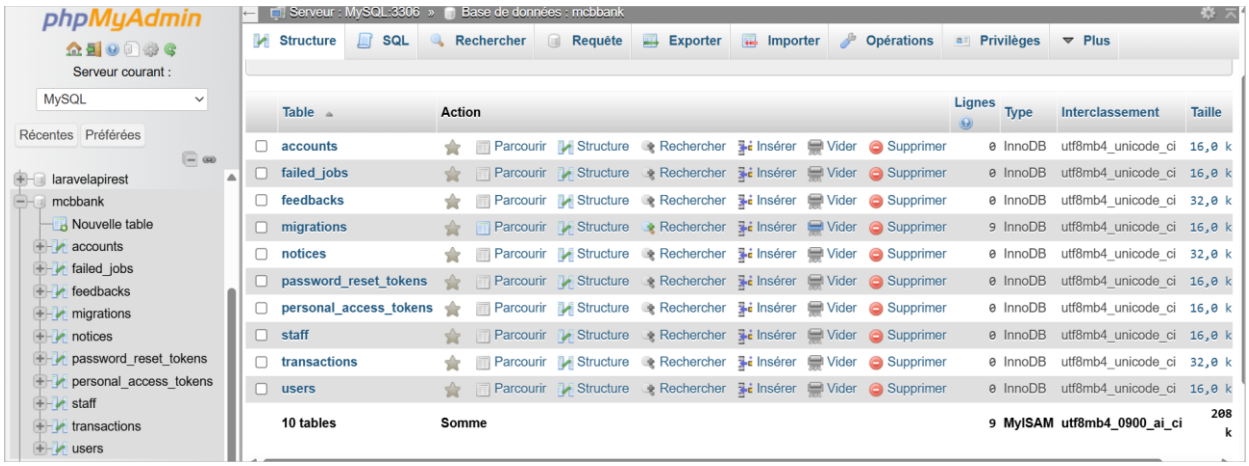


Figure 8 : *phpMyAdmin : base mcbbank avec les tables staff, accounts, transactions, feedbacks, notices*

2.4 Création du compte manager avec tinker

Il n'existe pas de page pour créer le manager : il est inséré depuis le terminal avec **tinker**.

```
php artisan tinker
DB::table('staff')->insert(['email' => 'admin@manager.com', 'password' => Hash::make('********'), 'type' =>
'manager', 'created_at' => now(), 'updated_at' => now()]);
exit
```

Le résultat est `= true`. Le mot de passe est crypté par `Hash::make()` : dans la base, il commence par `$2y$`.

```
PS C:\wamp64\www\MCBBank> php artisan tinker
Psy Shell v0.12.24 (PHP 8.2.18 – cli) by Justin Hileman
New PHP manual is available (latest: 3.1.4). Update with `doc --update-manual`
> DB::table('staff')->insert(['email' => 'admin@manager.com', 'password' => Hash::make(' ******** '), 'type' => 'manager', 'cre
ated_at' => now(), 'updated_at' => now()]);

= true
```

Figure 9 : *Terminal tinker : insertion du manager (= true), mot de passe masqué*

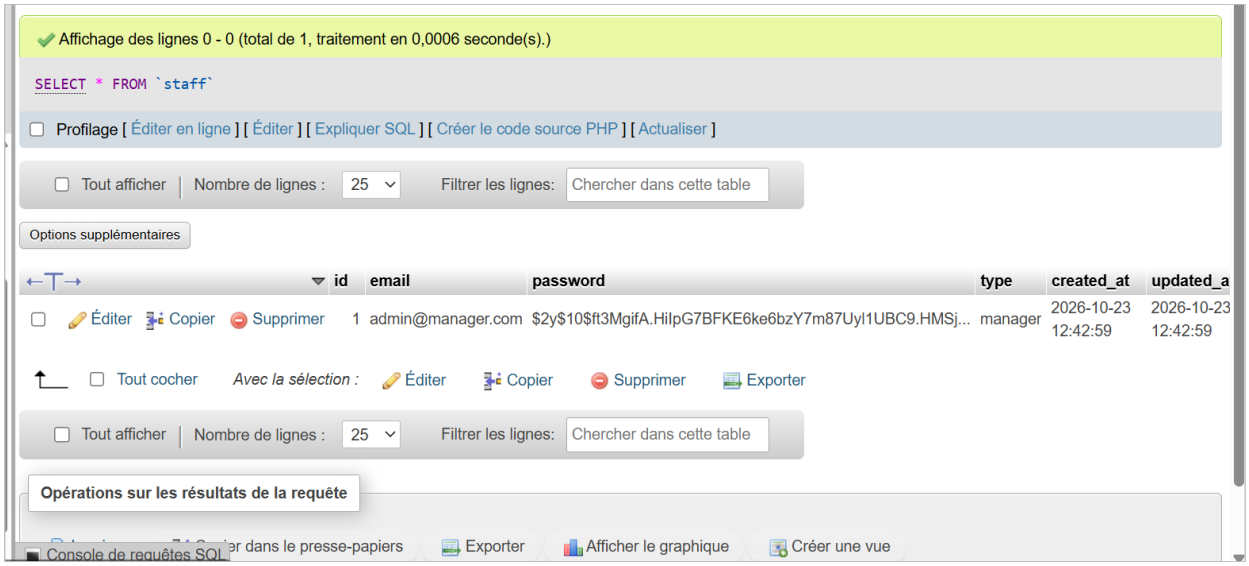


Figure 10 : *phpMyAdmin : table staff avec le mot de passe crypté*

3. Architecture de l'application

Les éléments communs à tout le site : contrôleurs, protection par rôle, routes, layouts Blade et connexion.

3.1 Création des contrôleurs

```
php artisan make:controller AuthController
php artisan make:controller AdminController
php artisan make:controller ClientController
php artisan make:controller CashierController
php artisan make:middleware CheckRole
```

Chaque contrôleur importe le Query Builder (`use Illuminate\Support\Facades\DB;`). `AuthController` et `AdminController` importent aussi `Hash` pour crypter et vérifier les mots de passe.

3.2 Middleware CheckRole : protéger les pages par rôle

Un client ne doit pas pouvoir ouvrir `/admin/home`, et une personne non connectée doit être renvoyée vers `/login`. Le middleware vérifie le rôle enregistré dans la session. Pour un client, il partage aussi ses informations avec toutes les vues (solde affiché dans la barre de navigation).

app/Http/Middleware/CheckRole.php

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use Symfony\Component\HttpFoundation\Response;

class CheckRole
{
    // Laisse passer seulement si l'utilisateur connecté a le bon rôle (manager, cashier ou client)
    public function handle(Request $request, Closure $next, string $role): Response
    {
        if (session('role') != $role) {
            return redirect('/login')->with('error', 'Veuillez vous connecter');
        }

        // Pour le client : on partage ses infos avec toutes les vues (solde dans la navbar)
        if ($role == 'client') {
            $client = DB::table('accounts')->where('id', session('client_id'))->first();
            view()->share('client', $client);
        }

        return $next($request);
    }
}
```

Le projet étant en **Laravel 10**, le middleware est déclaré sous le nom `role` dans `app/Http/Kernel.php` (en Laravel 11/12, c'est dans `bootstrap/app.php`) :

app/Http/Kernel.php

```php
    protected $middlewareAliases = [
        // MCB Bank : protège les pages selon le rôle (role:manager, role:client, role:cashier)
        'role' => \App\Http\Middleware\CheckRole::class,
```

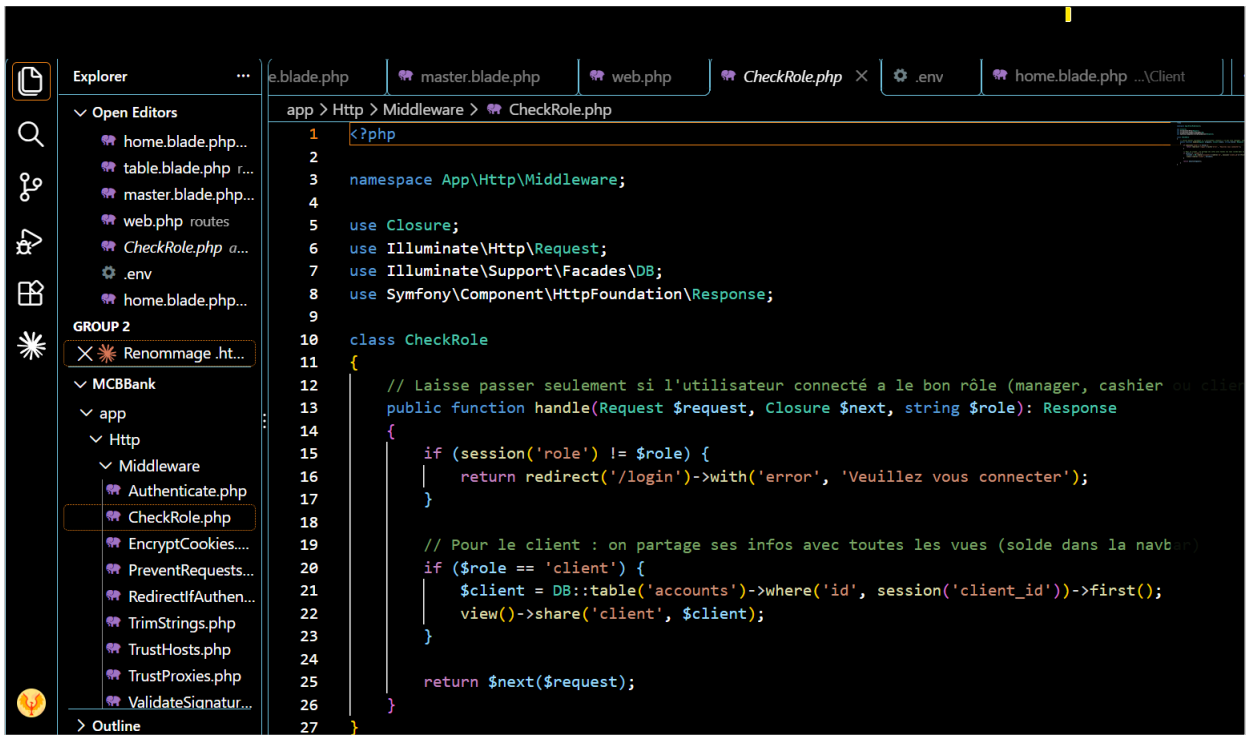


Figure 11 : *app/Http/Middleware/CheckRole.php*

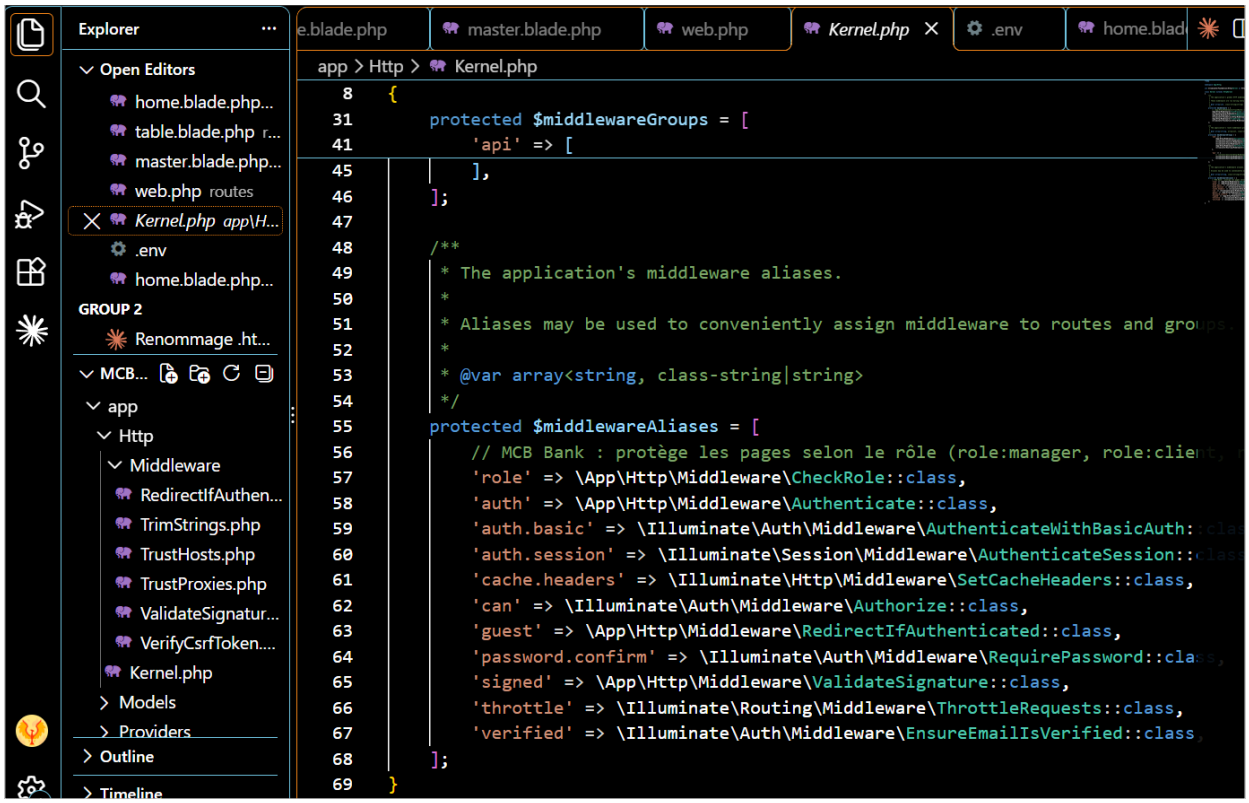


Figure 12 : *app/Http/Kernel.php : déclaration de l'alias role*

3.3 Les routes : web.php

Toutes les routes du site sont regroupées par rôle avec `middleware('role:...')`. Les routes `/admin/home` et `/client/home` demandées dans l'énoncé y figurent.

`routes/web.php`

> **À retenir :** Un formulaire HTML ne sait envoyer que GET et POST. Pour DELETE et PUT, on écrit `method="POST"` puis `@method('DELETE')` ou `@method('PUT')` dans le formulaire.

```
php artisan route:list --except-vendor
```

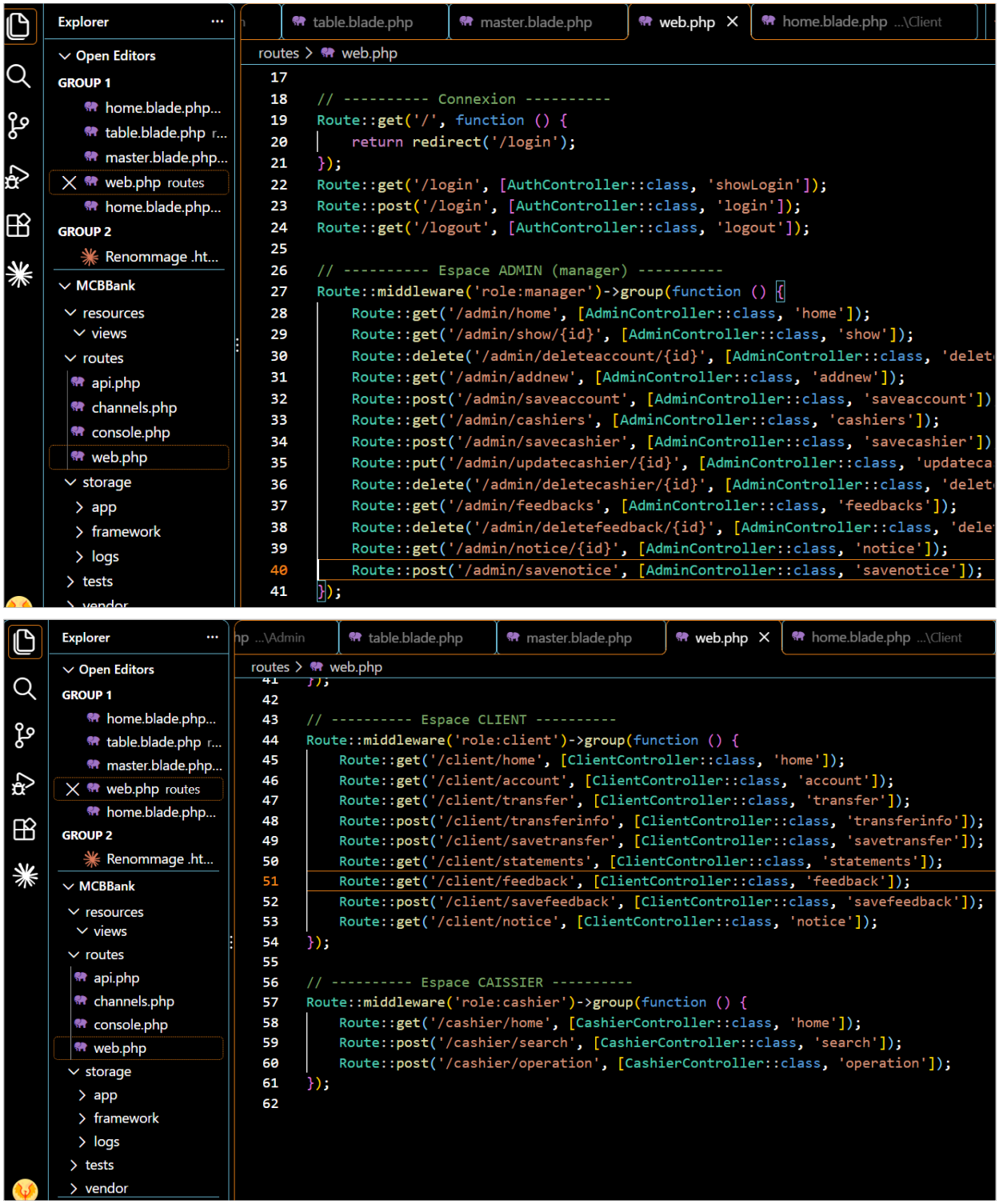


Figure 13 : *routes/web.php : routes de connexion, espace Admin, espace Client et espace Caissier*

3.4 Blade templating : layouts et barres de navigation

Toutes les pages du template répétaient le même `<head>` (CSS + JS) et la même barre de navigation. Ce code commun a été factorisé dans deux layouts :

Fichier	Rôle
`AdminLayout/master.blade.php`	squelette des pages Manager et Caissier
`AdminLayout/header.blade.php`	barre de navigation du Manager
`AdminLayout/sidebar.blade.php`	barre de navigation du Caissier
`ClientLayout/master.blade.php`	squelette des pages Client
`ClientLayout/header.blade.php`	barre de navigation du Client
`messages.blade.php`	messages de succès, d'erreur et de validation

Le layout master

Le layout contient le `<head>`, inclut la bonne barre de navigation selon le rôle connecté (`@include`), puis les messages, et réserve la zone de contenu avec `@yield('content')`.

`resources/views/AdminLayout/master.blade.php`

```
<!DOCTYPE html>
<html>
  <head>
    {{-- Titre de l'onglet : chaque page peut le personnaliser avec @section('title') --}}
    <title>@yield('title', 'Banking') - MCB Bank</title>

    {{-- asset() génère l'URL vers le dossier public/ --}}
    <link rel="stylesheet" type="text/css" href="{{ asset('frontend/css/bootstrap.min.css') }}">
    <link rel="stylesheet" type="text/css" href="{{ asset('frontend/css/custom.css') }}">
    <link rel="stylesheet" type="text/css" href="{{ asset('frontend/font-awesome/css/font-awesome.min.css')
}}">
    <script src="{{ asset('frontend/js/jquery-3.2.1.min.js') }}"></script>
    <script src="{{ asset('frontend/js/popper.min.js') }}"></script>
    <script src="{{ asset('frontend/js/bootstrap.min.js') }}"></script>
    <script type="text/javascript">
      // Active les info-bulles Bootstrap
      $(function () {
        $('[data-toggle="tooltip"]').tooltip()
      });
    </script>
  </head>
  <body style="background:#96D678;background-size: 100%">
    {{-- Barre de navigation selon le rôle connecté : Caissier (sidebar) ou Manager (header) --}}
    @if (session('role') == 'cashier')
      @include('AdminLayout.sidebar')
    @else
      @include('AdminLayout.header')
    @endif

    {{-- Messages de succès / d'erreur --}}
    @include('messages')

    {{-- Contenu propre à chaque page --}}
    @yield('content')

    {{-- Fenêtres modales éventuelles (ex : ajout / modification caissier) --}}
    @yield('modals')
  </body>
</html>
```

Chaque page n'a plus qu'à hériter du layout et remplir ses sections :

```
@extends('AdminLayout.master')
```

```
@section('title', 'Home')

@section('content')
  ... contenu propre à la page ...
@endsection
```

Les messages

resources/views/messages.blade.php

```
{{-- Affichage des messages : succès (status), erreur (error) et erreurs de validation --}}
{{-- Inclus dans AdminLayout.master, ClientLayout.master et login --}}
<div class="container">
  @if (session('status'))
    <div class="alert alert-success">{{ session('status') }}</div>
  @endif
  @if (session('error'))
    <div class="alert alert-danger">{{ session('error') }}</div>
  @endif
  @if ($errors->any())
    <div class="alert alert-danger">
      <ul class="mb-0">
        @foreach ($errors->all() as $error)
          <li>{{ $error }}</li>
        @endforeach
      </ul>
    </div>
  @endif
</div>
```

Les barres de navigation

Les liens `.html` du template ont été remplacés par `{{ url('/chemin') }}`. Le menu actif est géré avec `request()->is(...)`.

resources/views/ClientLayout/header.blade.php (extrait)

```
<ul class="navbar-nav mr-auto">
      <li class="nav-item">
        <a class="nav-link {{ request()->is('client/home') ? 'active' : '' }}" href="{{ url('/client/home')
}}">Home</a>
      </li>
      <li class="nav-item">
        <a class="nav-link {{ request()->is('client/account') ? 'active' : '' }}" href="{{
url('/client/account') }}">Accounts</a>
      </li>
      <li class="nav-item">
        <a class="nav-link {{ request()->is('client/statements') ? 'active' : '' }}" href="{{
url('/client/statements') }}">Account Statements</a>
      </li>
      <li class="nav-item">
        <a class="nav-link {{ request()->is('client/transfer*') ? 'active' : '' }}" href="{{
url('/client/transfer') }}">Funds Transfer</a>
      </li>
    </ul>
    <form class="form-inline my-2 my-lg-0">
      {{-- $client est envoyé par le middleware CheckRole --}}
      <a href="{{ url('/client/account') }}" class="btn btn-outline-success" data-toggle="tooltip"
title="Your current Account Balance">Account Balance : Rs.{{ $client->balance }}</a>
      <a href="{{ url('/client/account') }}" data-toggle="tooltip" title="Account Summary" class="btn btn-
outline-primary mx-1"><i class="fa fa-book fa-fw"></i></a>
      <a href="{{ url('/client/notice') }}" data-toggle="tooltip" title="View Notice" class="btn btn-
outline-primary mx-1"><i class="fa fa-envelope fa-fw"></i></a>
      <a href="{{ url('/client/feedback') }}" data-toggle="tooltip" title="Help?" class="btn btn-outline-
info mx-1"><i class="fa fa-question fa-fw"></i></a>
      <a href="{{ url('/logout') }}" data-toggle="tooltip" title="Logout" class="btn btn-outline-danger mx-
1"><i class="fa fa-sign-out fa-fw"></i></a>
    </form>
```

resources/views/AdminLayout/master.blade.php

```
<body style="background:#96D678;background-size: 100%">
  {{-- Barre de navigation selon le rôle connecté : Caissier (sidebar) ou Manager (header) --}}
  @if (session('role') == 'cashier')
    @include('AdminLayout.sidebar')
  @else
    @include('AdminLayout.header')
  @endif

  {{-- Messages de succès / d'erreur --}}
  @include('messages')

  {{-- Contenu propre à chaque page --}}
  @yield('content')

  {{-- Fenêtres modales éventuelles (ex : ajout / modification caissier) --}}
  @yield('modals')
</body>
```

Figure 14 : *AdminLayout/master.blade.php : @include et @yield*

resources/views/ClientLayout/header.blade.php

```
<li class="nav-item">
  <a class="nav-link {{ request()->is('client/home') ? 'active' : '' }}"
     href="{{ url('/client/home') }}">Home</a>
</li>
<li class="nav-item">
  <a class="nav-link {{ request()->is('client/transfer*') ? 'active' : '' }}"
     href="{{ url('/client/transfer') }}">Funds Transfer</a>
</li>
...
{{-- $client est envoyé par le middleware CheckRole --}}
<a href="{{ url('/client/account') }}" class="btn btn-outline-success"
   title="Your current Account Balance">Account Balance : Rs.{{ $client->balance }}</a>
<a href="{{ url('/client/notice') }}" class="btn btn-outline-primary mx-1"><i class="fa fa-envelope fa-fw"></i></a>
<a href="{{ url('/logout') }}" class="btn btn-outline-danger mx-1"><i class="fa fa-sign-out fa-fw"></i></a>
```

Figure 15 : *ClientLayout/header.blade.php : liens url() et solde du client*

3.5 Connexion et déconnexion

La page de connexion contient trois formulaires avec trois boutons différents : `userLogin`, `managerLogin` et `cashierLogin`. Le contrôleur regarde quel bouton a été cliqué avec `$request->has(...)`.

Dans AuthController

```
// Affiche la page de connexion
public function showLogin()
{
    return view('login');
}
```

```
// Vérifie l'email et le mot de passe selon le bouton cliqué
public function login(Request $request)
{
    $request->validate([
        'email' => 'required|email',
        'password' => 'required',
    ]);

    // Connexion Client
    if ($request->has('userLogin')) {
        $account = DB::table('accounts')->where('email', $request->email)->first();
        if ($account && Hash::check($request->password, $account->password)) {
            session(['role' => 'client', 'client_id' => $account->id]);
            return redirect('/client/home');
        }
    }

    // Connexion Manager (admin) ou Caissier : table staff
    if ($request->has('managerLogin') || $request->has('cashierLogin')) {
        $type = $request->has('managerLogin') ? 'manager' : 'cashier';
        $staff = DB::table('staff')->where('email', $request->email)->where('type', $type)->first();
```

```
        if ($staff && Hash::check($request->password, $staff->password)) {
            session(['role' => $type, 'staff_id' => $staff->id]);
            if ($type == 'manager') {
                return redirect('/admin/home');
            }
            return redirect('/cashier/home');
        }
    }

    return back()->with('error', 'Email ou mot de passe incorrect');
}
```

```
// Déconnexion : on vide la session
public function logout()
{
    session()->flush();
    return redirect('/login');
}
```



Figure 16 : *AuthController : méthode login() dans VS Code*

Dans la vue login.blade.php

Chaque formulaire envoie en POST vers `/login` avec `@csrf`. Sans `@csrf`, Laravel répond « 419 Page Expired ».

resources/views/login.blade.php (extrait)

```
{{-- 2) Connexion Manager --}}
      <div class="card rounded-0">
        <div class="card-header" role="tab" id="headingTwo">
          <h5 class="mb-0">
            <a class="collapsed btn btn-outline-success btn-block" data-toggle="collapse"
href="#collapseTwo" aria-expanded="false" aria-controls="collapseTwo">Manager Login</a>
          </h5>
        </div>
        <div id="collapseTwo" class="collapse" role="tabpanel" aria-labelledby="headingTwo" data-
parent="#accordion">
          <div class="card-body">
            <form method="POST" action="{{ url('/login') }}">
              @csrf
              <input type="email" name="email" class="form-control" required placeholder="Enter Email">
              <input type="password" name="password" class="form-control" required placeholder="Enter
Password">
```

```
            <button type="submit" class="btn btn-primary btn-block btn-sm my-1"
name="managerLogin">Enter</button>
          </form>
```

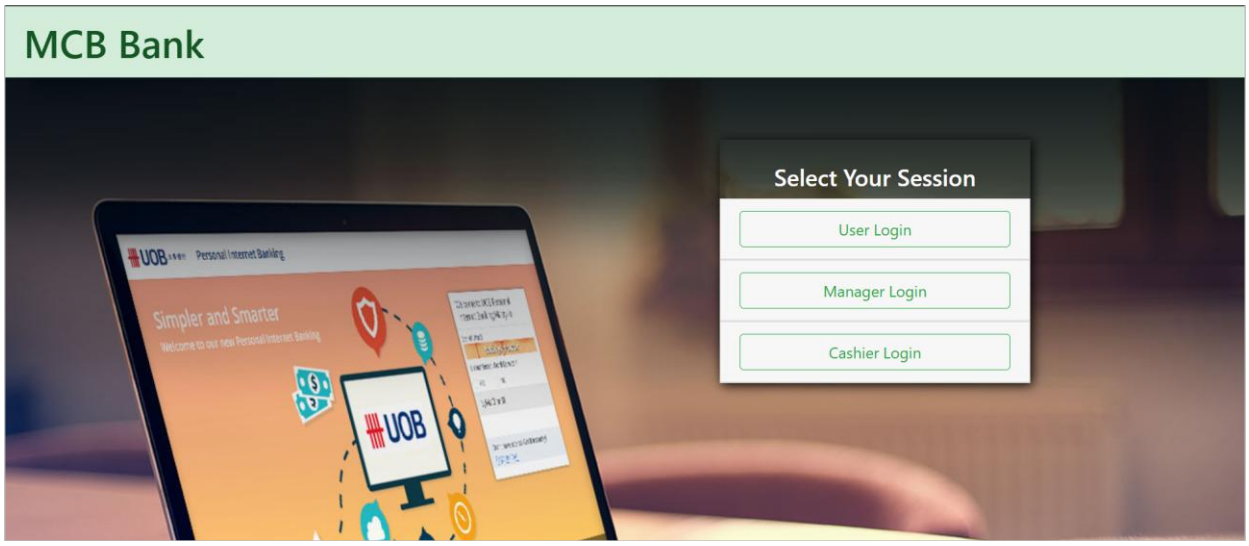


Figure 17 : *Page de connexion (redirection de / vers /login)*

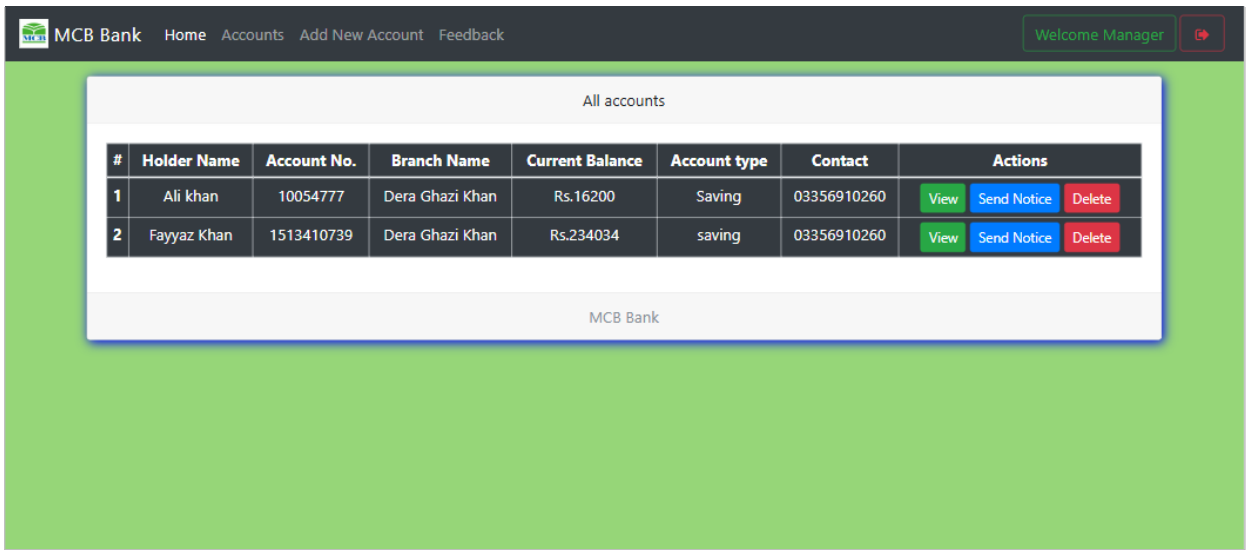


Figure 18 : *Connexion manager réussie : page /admin/home*

Dans le navigateur : après un clic sur Logout, ouvrir http://127.0.0.1:8000/admin/home renvoie vers /login.

. Espace Manager (/admin)

Toutes les fonctionnalités accessibles après une connexion en tant que manager.

4.1 Page d'accueil : liste des comptes clients

AdminController

```
// Page d'accueil admin : liste de tous les comptes clients
public function home()
{
    $accounts = DB::table('accounts')->orderBy('name', 'asc')->get();
    // Eloquent : $accounts = Account::orderBy('name', 'asc')->get();
    return view('Admin.home', ['accounts' => $accounts]);
}
```

Dans la vue, les lignes écrites à la main du template sont remplacées par une boucle `@forelse` :

resources/views/Admin/home.blade.php (extrait)

```
@forelse ($accounts as $account)
            <tr>
              <th scope="row">{{ $loop->iteration }}</th>
              <td>{{ $account->name }}</td>
              <td>{{ $account->accountNo }}</td>
              <td>{{ $account->city }}</td>
              <td>Rs.{{ $account->balance }}</td>
              <td>{{ $account->accountType }}</td>
              <td>{{ $account->number }}</td>
              <td>
                <a href="{{ url('/admin/show/'.$account->id) }}" class="btn btn-success btn-sm" data-
toggle="tooltip" title="View More info">View</a>
                <a href="{{ url('/admin/notice/'.$account->id) }}" class="btn btn-primary btn-sm" data-
toggle="tooltip" title="Send notice to this">Send Notice</a>
                {{-- Suppression : formulaire avec la méthode DELETE --}}
                <form action="{{ url('/admin/deleteaccount/'.$account->id) }}" method="POST"
style="display:inline">
                  @csrf
                  @method('DELETE')
                  <button type="submit" class="btn btn-danger btn-sm" onclick="return confirm('Supprimer ce
compte ?')">Delete</button>
                </form>
              </td>
            </tr>
          @empty
            <tr><td colspan="8">Aucun compte pour le moment</td></tr>
          @endforelse
```

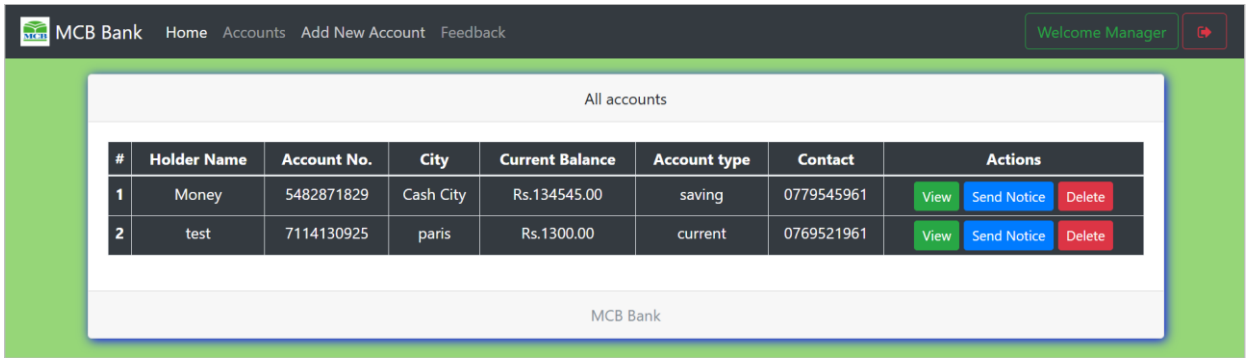


Figure 19 : */admin/home : liste des comptes clients*

4.2 Création d'un compte bancaire

`addnew()` affiche le formulaire avec un numéro de compte généré automatiquement. `saveaccount()` valide les données côté serveur, enregistre la photo, puis insère le compte avec le Query Builder.

AdminController

```
// Formulaire de création de compte bancaire
public function addnew()
{
    $accountNo = rand(1000000000, 9999999999); // numéro de compte généré automatiquement
    return view('Admin.maddnew', ['accountNo' => $accountNo]);
}
```

```
// Enregistrer le nouveau compte bancaire
public function saveaccount(Request $request)
{
    $request->validate([
        'name' => 'required',
        'cnic' => 'required',
        'accountNo' => 'required|unique:accounts',
        'accountType' => 'required',
        'city' => 'required',
        'address' => 'required',
        'email' => 'required|email|unique:accounts',
        'password' => 'required|min:4',
        'balance' => 'required|numeric|min:500',
        'source' => 'required',
        'number' => 'required',
        'image' => 'required|image',
    ]);

    // La photo est enregistrée dans storage/app/public/photos
    $image = $request->file('image')->store('photos', 'public');

    DB::table('accounts')->insert([
        'name' => $request->name,
        'cnic' => $request->cnic,
        'accountNo' => $request->accountNo,
        'accountType' => $request->accountType,
        'city' => $request->city,
        'address' => $request->address,
        'email' => $request->email,
        'password' => Hash::make($request->password),
        'balance' => $request->balance,
        'source' => $request->source,
        'number' => $request->number,
        'image' => $image,
        'created_at' => now(),
        'updated_at' => now(),
    ]);

    return redirect('/admin/home')->with('status', 'Le compte de '.$request->name.' a été créé avec
succès');
}
```

Le formulaire a besoin de `action`, de `@csrf` et de `enctype="multipart/form-data"` pour envoyer la photo. `old('name')` conserve la saisie si la validation échoue.

resources/views/Admin/maddnew.blade.php (extrait)

```
{{-- enctype obligatoire pour envoyer la photo --}}
      <form method="POST" action="{{ url('/admin/saveaccount') }}" enctype="multipart/form-data">
        @csrf
        <table class="table">
          <tbody>
            <tr>
              <th>Name</th>
              <td><input type="text" name="name" value="{{ old('name') }}" class="form-control input-sm"
required></td>
              <th>CNIC</th>
              <td><input type="number" name="cnic" value="{{ old('cnic') }}" class="form-control input-sm"
required></td>
            </tr>
```

Pour que les photos soient visibles dans le navigateur, on crée une seule fois le lien `public/storage` :

```
php artisan storage:link
```

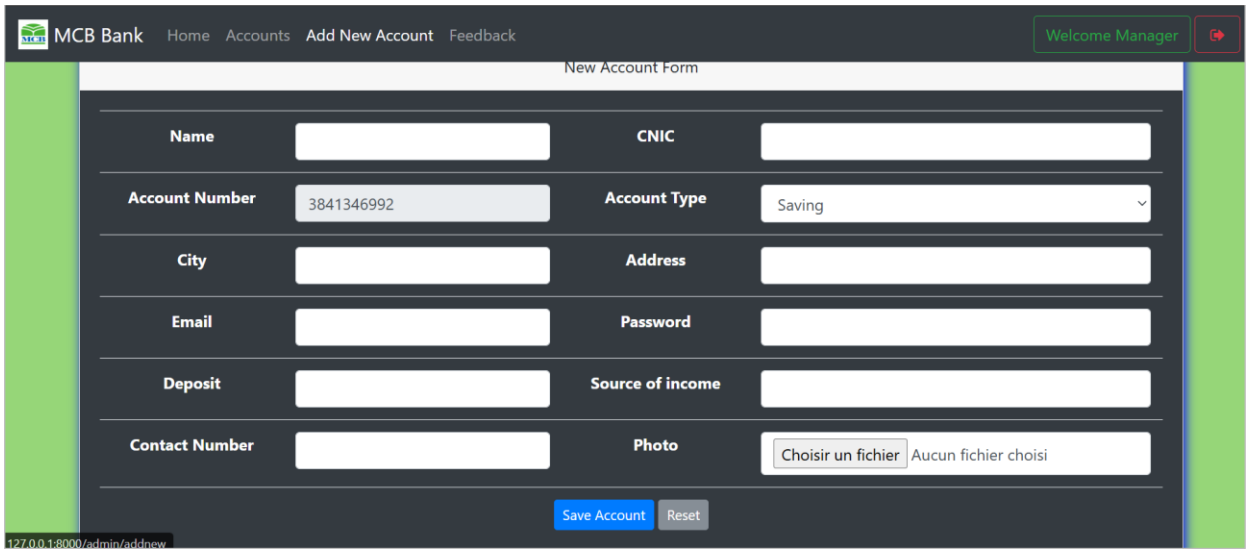


Figure 20 : *Formulaire « New Account Form » (numéro de compte généré automatiquement)*

4.3 Affichage des détails d'un client

Le bouton **View** de la page d'accueil pointe vers `/admin/show/{id}`.

AdminController

```
// Détails d'un client
public function show($id)
{
    $account = DB::table('accounts')->where('id', $id)->first();
    // Eloquent : $account = Account::find($id);
    return view('Admin.mshow', ['account' => $account]);
}
```

resources/views/Admin/mshow.blade.php (extrait)

```
<div class="text-center">
        {{-- Photo enregistrée dans storage/app/public/photos --}}
        <img src="{{ asset('storage/'.$account->image) }}" height="100" width="100" alt="" class="rounded-
circle m-2" style="border : 2px solid #FFF;">
      </div>
      <table class="table table-bordered">
        <tbody>
          <tr>
            <td>Name</td><th>{{ $account->name }}</th>
            <td>Account No</td><th>{{ $account->accountNo }}</th>
          </tr>
          <tr>
            <td>Email</td><th>{{ $account->email }}</th>
            <td>Source of income</td><th>{{ $account->source }}</th>
          </tr>
          <tr>
            <td>Current Balance</td><th>Rs.{{ $account->balance }}</th>
            <td>Account Type</td><th>{{ $account->accountType }}</th>
          </tr>
          <tr>
            <td>Cnic</td><th>{{ $account->cnic }}</th>
            <td>City</td><th>{{ $account->city }}</th>
          </tr>
          <tr>
            <td>Contact Number</td><th>{{ $account->number }}</th>
            <td>Address</td><th>{{ $account->address }}</th>
          </tr>
        </tbody>
```

4.4 Suppression d'un compte client

Le bouton Delete de la liste des comptes (voir le formulaire @method('DELETE') en 4.1) envoie une requête DELETE vers /admin/deleteaccount/{id}. Grâce à cascadeOnDelete(), les transactions, feedbacks et notices du client sont supprimés avec lui.

AdminController

```
// Supprimer un compte client
public function deleteaccount($id)
{
    DB::table('accounts')->where('id', $id)->delete();
    // Eloquent : Account::find($id)->delete();
    return redirect('/admin/home')->with('status', 'Le compte a été supprimé avec succès');
}
```

4.5 Gestion des caissiers (CRUD)

La page `maccounts` contient la liste des caissiers (**Read**), une fenêtre modale « New Cashier Account » (**Create**), une fenêtre modale « Edit » par caissier (**Update**), et un bouton **Delete**.

AdminController

```
// Liste des caissiers
public function cashiers()
{
    $cashiers = DB::table('staff')->where('type', 'cashier')->get();
    return view('Admin.maccounts', ['cashiers' => $cashiers]);
}
```

```
// Ajouter un caissier
public function savecashier(Request $request)
{
    $request->validate([
        'email' => 'required|email|unique:staff',
        'password' => 'required|min:4',
    ]);

    DB::table('staff')->insert([
        'email' => $request->email,
        'password' => Hash::make($request->password),
        'type' => 'cashier',
        'created_at' => now(),
        'updated_at' => now(),
    ]);

    return redirect('/admin/cashiers')->with('status', 'Caissier ajouté avec succès');
}
```

```
// Modifier un caissier
public function updatecashier(Request $request, $id)
{
    $request->validate([
        'email' => 'required|email|unique:staff,email,'.$id,
        'password' => 'required|min:4',
    ]);

    DB::table('staff')->where('id', $id)->update([
        'email' => $request->email,
        'password' => Hash::make($request->password),
        'updated_at' => now(),
    ]);

    return redirect('/admin/cashiers')->with('status', 'Caissier modifié avec succès');
}
```

```
// Supprimer un caissier
public function deletecashier($id)
{
    DB::table('staff')->where('id', $id)->delete();
    return redirect('/admin/cashiers')->with('status', 'Caissier supprimé avec succès');
}
```

À retenir : `'unique:staff,email,'.$id` : à la modification, le caissier a le droit de garder son propre email.

resources/views/Admin/maccounts.blade.php (extrait : modale Update)

```
{{-- Modales : modification d'un caissier (Update), une par caissier pour pré-remplir son email --}}
@foreach ($cashiers as $cashier)
<div class="modal fade" id="editModal{{ $cashier->id }}" tabindex="-1" role="dialog" aria-hidden="true">
  <div class="modal-dialog" role="document">
    <div class="modal-content bg-dark text-white">
      <div class="modal-header">
        <h5 class="modal-title">Edit Cashier Account</h5>
        <button type="button" class="close" data-dismiss="modal" aria-label="Close">
          <span aria-hidden="true">&times;</span>
        </button>
      </div>
      <div class="modal-body">
        <form method="POST" action="{{ url('/admin/updatecashier/'.$cashier->id) }}">
          @csrf
          @method('PUT')
          <div class="form-group">
            <input class="form-control w-75 mx-auto" type="email" name="email" value="{{ $cashier->email
}}" required placeholder="Email">
          </div>
          <div class="form-group">
            <input class="form-control w-75 mx-auto" type="password" name="password" required
placeholder="New Password">
          </div>
          <div class="modal-footer">
            <button type="button" class="btn btn-secondary" data-dismiss="modal">Close</button>
            <button type="submit" class="btn btn-primary">Update Account</button>
          </div>
        </form>
```

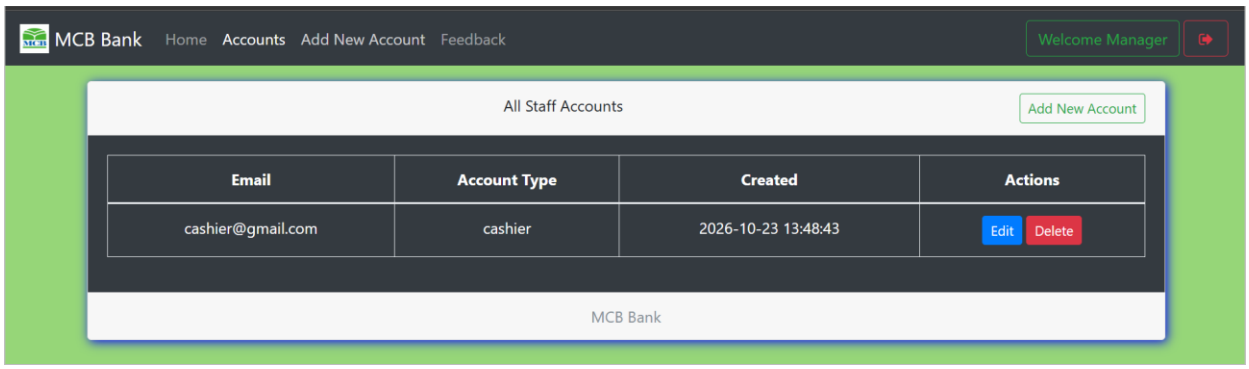


Figure 21 : *Liste des caissiers (All Staff Accounts)*

4.6 Envoi d'une notice à un client

AdminController

```
// Formulaire pour envoyer une notice à un client
public function notice($id)
{
    $account = DB::table('accounts')->where('id', $id)->first();
    return view('Admin.mnotice', ['account' => $account]);
}
```

```
// Enregistrer la notice
public function savenotice(Request $request)
```

```
{
    $request->validate([
        'notice' => 'required',
        'userId' => 'required',
    ]);

    DB::table('notices')->insert([
        'account_id' => $request->userId,
        'notice' => $request->notice,
        'created_at' => now(),
        'updated_at' => now(),
    ]);

    return redirect('/admin/home')->with('status', 'Notice envoyée avec succès');
}
```

resources/views/Admin/mnotice.blade.php (extrait)

```
<form method="POST" action="{{ url('/admin/savenotice') }}">
        @csrf
        <div class="alert alert-success w-50 mx-auto">
          <h5>Write notice for {{ $account->name }}</h5>
          {{-- id du client destinataire --}}
          <input type="hidden" name="userId" value="{{ $account->id }}">
          <textarea class="form-control" name="notice" required placeholder="Write your
message"></textarea>
          <button type="submit" name="send" class="btn btn-primary btn-block btn-sm my-1">Send</button>
        </div>
      </form>
```

4.7 Consultation et suppression des feedbacks

Pour afficher le nom et le numéro du client à côté de son message, on fait une **jointure** entre `feedbacks` et `accounts` avec le Query Builder.

AdminController

```
// Liste des feedbacks avec le nom du client (jointure)
public function feedbacks()
{
    $feedbacks = DB::table('feedbacks')
        ->join('accounts', 'feedbacks.account_id', '=', 'accounts.id')
        ->select('feedbacks.*', 'accounts.name', 'accounts.accountNo', 'accounts.number')
        ->orderBy('feedbacks.created_at', 'desc')
        ->get();
    return view('Admin.mfeedback', ['feedbacks' => $feedbacks]);
}
```

```
// Supprimer un feedback
public function deletefeedback($id)
{
    DB::table('feedbacks')->where('id', $id)->delete();
    return redirect('/admin/feedbacks')->with('status', 'Feedback supprimé avec succès');
}
```

resources/views/Admin/mfeedback.blade.php (extrait)

```
@forelse ($feedbacks as $feedback)
            <tr>
              <td>{{ $feedback->name }}</td>
              <td>{{ $feedback->accountNo }}</td>
              <td>{{ $feedback->number }}</td>
              <td>{{ $feedback->message }}</td>
              <td>{{ $feedback->created_at }}</td>
              <td>
                <form action="{{ url('/admin/deletefeedback/'.$feedback->id) }}" method="POST">
                  @csrf
                  @method('DELETE')
                  <button type="submit" class="btn btn-danger btn-sm" onclick="return confirm('Supprimer ce
message ?')">Delete</button>
```

```
            </form>
          </td>
        </tr>
      @empty
        <tr><td colspan="6">Aucun feedback pour le moment</td></tr>
      @endforelse
```

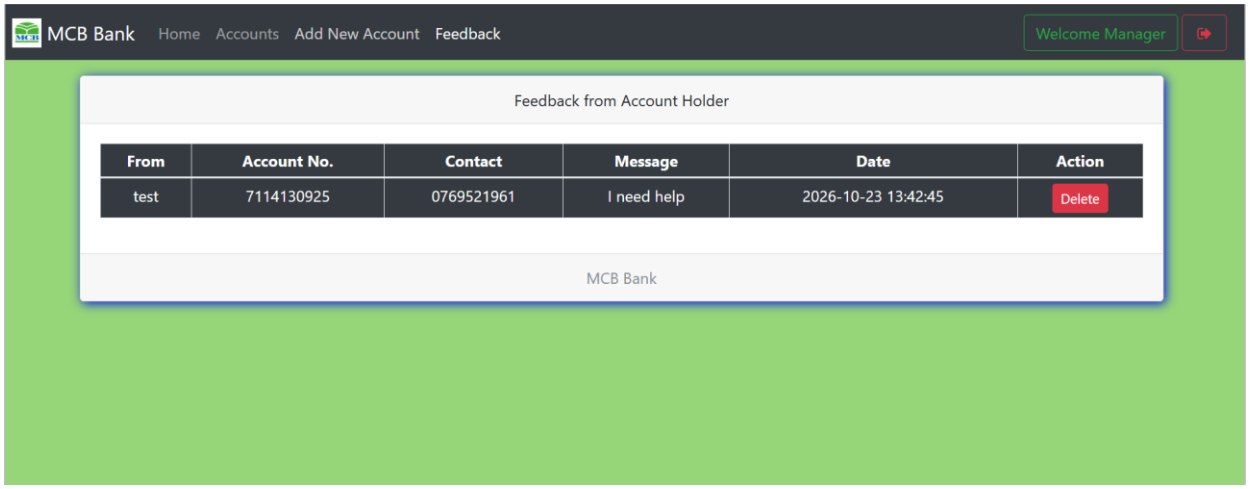


Figure 22 : */admin/feedbacks : message du client avec son nom*

5. Espace Client (/client)

Les pages accessibles au client connecté. Ses informations ($client) sont partagées avec toutes les vues par le middleware CheckRole.

5.1 Page d'accueil

ClientController

```
// Page d'accueil du client avec la dernière notice reçue
public function home()
{
    $notice = DB::table('notices')->where('account_id', session('client_id'))->latest()->first();
    return view('Client.home', ['notice' => $notice]);
}
```

La variable `$client` vient du middleware `CheckRole` ; `$notice` est la dernière notice reçue.

resources/views/Client/home.blade.php (extrait)

```
<div class="jumbotron shadowBlack" style="padding: 25px;min-height: 241px;max-height: 241px">
          {{-- $client : envoyé par le middleware CheckRole ; $notice : dernière notice
(ClientController@home) --}}
          <h4 class="display-5">Welcome {{ $client->name }} to MCB Bank</h4>
          <p  class="lead alert alert-warning"><b>Latest Notification:</b>
          {{ $notice ? $notice->notice : 'Aucune notification' }}</p>
        </div>
```

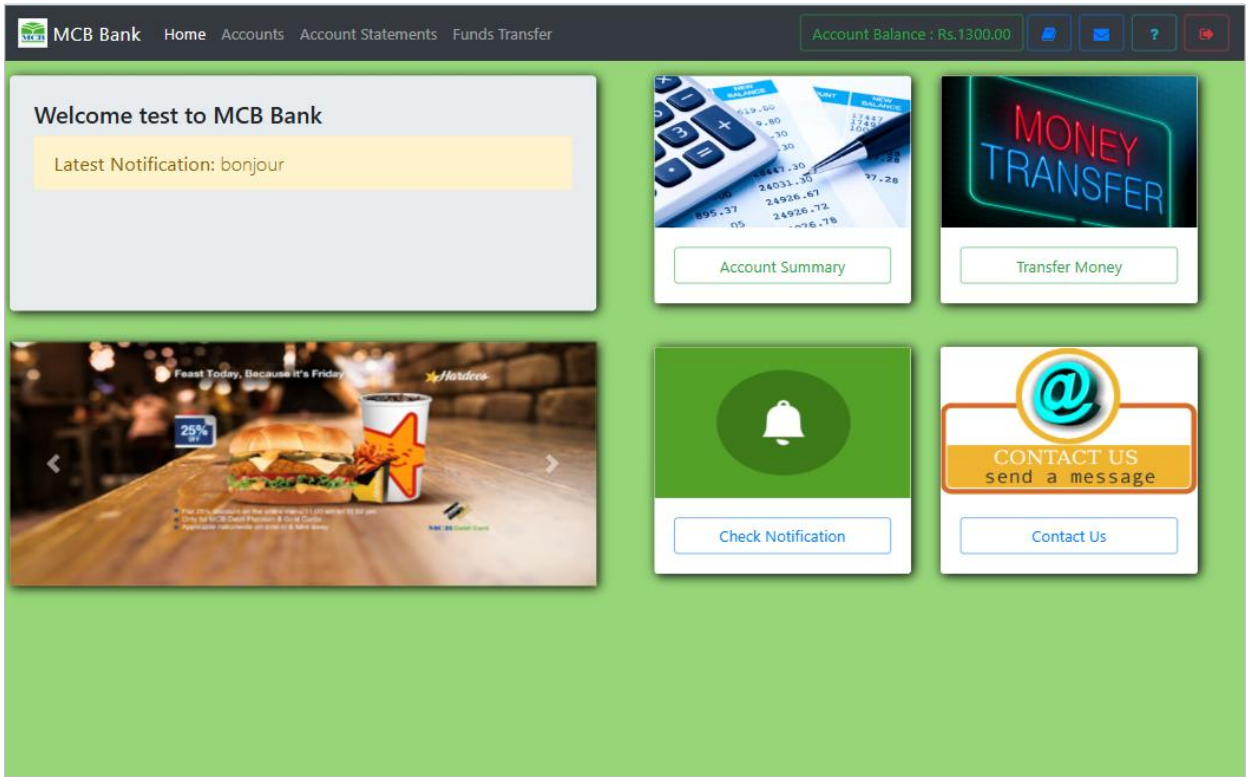


Figure 23 : */client/home : nom du client, dernière notice, solde dans la barre du haut*

Dans le navigateur : connecté en client, ouvrir /admin/home renvoie vers /login.

5.2 Détails du compte bancaire

ClientController

```
// Détails du compte bancaire
public function account()
{
```

```
    // $client est déjà envoyé à toutes les vues par le middleware CheckRole
    return view('Client.accounts');
}
```

resources/views/Client/accounts.blade.php (extrait)

```
<table class="table table-striped table-dark w-75 mx-auto">
        <thead>
          <tr>
            <td scope="col">Account No.</td>
            <th scope="col">{{ $client->accountNo }}</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <th scope="row">Holder Name</th>
            <td>{{ $client->name }}</td>
          </tr>
          <tr>
            <th scope="row">City</th>
            <td>{{ $client->city }}</td>
          </tr>
          <tr>
            <th scope="row">Account Type</th>
            <td>{{ $client->accountType }}</td>
          </tr>
          <tr>
            <th scope="row">Current Balance</th>
            <td>Rs.{{ $client->balance }}</td>
          </tr>
          <tr>
            <th scope="row">Account Created</th>
            <td>{{ $client->created_at }}</td>
          </tr>
        </tbody>
      </table>
```

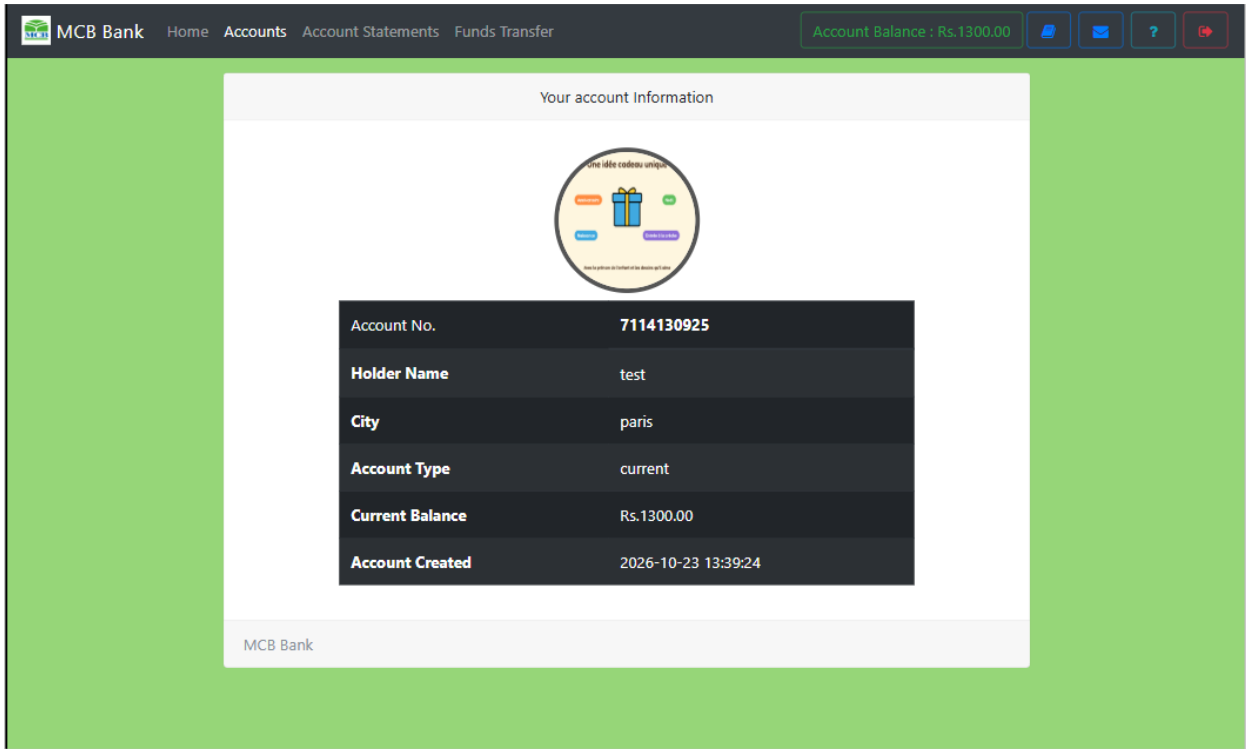


Figure 24 : */client/account : photo, numéro de compte, type, solde, date de création*

5.3 Transfert de fonds

Le template contient deux pages, `transfer` (saisir le numéro du destinataire) et `transfe` (confirmer et saisir le montant). Le transfert se fait donc en 3 étapes :

1. `GET /client/transfer` : formulaire du numéro de compte et historique ;
2. `POST /client/transferinfo` : recherche du destinataire, affichage de ses informations et du champ montant ;
3. `POST /client/savetransfer` : débit de l'émetteur, crédit du destinataire, enregistrement de 2 transactions.

ClientController

```
// Transfert - étape 1 : saisir le numéro de compte du destinataire
public function transfer()
{
    return view('Client.transfer', ['history' => $this->history()]);
}
```

```
// Transfert - étape 2 : afficher les infos du destinataire et saisir le montant
public function transferinfo(Request $request)
{
    $request->validate(['otherNo' => 'required']);

    $receiver = DB::table('accounts')->where('accountNo', $request->otherNo)->first();
    if (!$receiver) {
        return redirect('/client/transfer')->with('error', 'Aucun compte ne correspond à ce numéro');
    }
    if ($receiver->id == session('client_id')) {
        return redirect('/client/transfer')->with('error', 'Vous ne pouvez pas transférer vers votre propre
compte');
    }

    return view('Client.transfe', ['receiver' => $receiver, 'history' => $this->history()]);
}
```

```
// Transfert - étape 3 : effectuer le transfert
public function savetransfer(Request $request)
{
    $request->validate([
        'otherNo' => 'required|exists:accounts,accountNo',
        'amount' => 'required|numeric|min:1',
    ]);

    $me = DB::table('accounts')->where('id', session('client_id'))->first();
    $receiver = DB::table('accounts')->where('accountNo', $request->otherNo)->first();

    if ($me->balance < $request->amount) {
        return redirect('/client/transfer')->with('error', 'Solde insuffisant');
    }

    // DB::transaction : soit tout est fait, soit rien (pas d'argent perdu en cas d'erreur)
    DB::transaction(function () use ($me, $receiver, $request) {
        DB::table('accounts')->where('id', $me->id)->decrement('balance', $request->amount);
        DB::table('accounts')->where('id', $receiver->id)->increment('balance', $request->amount);

        // Une ligne "débit" pour l'émetteur, une ligne "crédit" pour le destinataire
        DB::table('transactions')->insert([
            'account_id' => $me->id,
            'action' => 'transfer',
            'debit' => $request->amount,
            'other' => $receiver->accountNo,
            'created_at' => now(),
            'updated_at' => now(),
        ]);
        DB::table('transactions')->insert([
            'account_id' => $receiver->id,
            'action' => 'transfer',
            'credit' => $request->amount,
            'other' => $me->accountNo,
            'created_at' => now(),
            'updated_at' => now(),
        ]);
    });
```

```
    return redirect('/client/transfer')->with('status', 'Transfert de Rs.'.$request->amount.' effectué avec
succès');
}
```

```
// Historique des transferts envoyés par le client
private function history()
{
    return DB::table('transactions')
        ->where('account_id', session('client_id'))
        ->where('action', 'transfer')
        ->whereNotNull('debit')
        ->orderBy('created_at', 'desc')
        ->get();
}
```

À retenir : `decrement()` et `increment()` enlèvent ou ajoutent un montant à une colonne. `DB::transaction()` garantit que, si une requête échoue, rien n'est modifié.

resources/views/Client/transfe.blade.php (extrait)

```
<form method="POST" action="{{ url('/client/savetransfer') }}">
          @csrf
          Account No.
          <input type="text" value="{{ $receiver->accountNo }}" name="otherNo" class="form-control"
readonly required>
          Account Holder Name.
          <input type="text" class="form-control" value="{{ $receiver->name }}" readonly>
          Account Holder Bank Name.
          <input type="text" class="form-control" value="MCB Bank" readonly>
          Enter Amount for transfer.
          {{-- max = solde du client connecté --}}
          <input type="number" name="amount" class="form-control" min="1" max="{{ $client->balance }}"
required>
          <button type="submit" name="transferSelf" class="btn btn-primary btn-sm my-1">Transfer</button>
          <a href="{{ url('/client/transfer') }}" class="btn btn-secondary btn-sm my-1">Cancel</a>
        </form>
```

5.4 Relevé bancaire

ClientController

```
// Relevé bancaire : toutes les opérations du client, 10 par page
public function statements()
{
    $transactions = DB::table('transactions')
        ->where('account_id', session('client_id'))
        ->orderBy('created_at', 'desc')
        ->paginate(10);
    return view('Client.statements', ['transactions' => $transactions]);
}
```

Un message différent est affiché selon le type d'opération, et `paginate(10)` affiche 10 opérations par page :

resources/views/Client/statements.blade.php (extrait)

```
@forelse ($transactions as $t)
        @if ($t->action == 'withdraw')
          <div class="alert alert-secondary">You withdraw Rs.{{ $t->debit }} from your account at {{ $t-
>created_at }}</div>
        @elseif ($t->action == 'deposit')
          <div class="alert alert-success">You deposit Rs.{{ $t->credit }} in your account at {{ $t-
>created_at }}</div>
        @elseif ($t->debit)
          <div class="alert alert-warning">You transfer Rs.{{ $t->debit }} to account no.{{ $t->other }} at
{{ $t->created_at }}</div>
        @else
```

```
          <div class="alert alert-info">You received Rs.{{ $t->credit }} from account no.{{ $t->other }} at
{{ $t->created_at }}</div>
        @endif
      @empty
        <div class="alert alert-secondary">Aucune opération pour le moment</div>
      @endforelse


      {{-- Liens de pagination --}}
      {{ $transactions->links() }}
```

Attention : Laravel 10 dessine la pagination en Tailwind par défaut. Comme le template utilise Bootstrap 4, on ajoute `Paginator::useBootstrapFour();` dans la méthode `boot()` de `app/Providers/AppServiceProvider.php`.

5.5 Envoi d'un feedback à la banque

ClientController

```
// Formulaire de feedback
public function feedback()
{
    return view('Client.feedback');
}
```

```
// Enregistrer le feedback
public function savefeedback(Request $request)
{
    $request->validate(['msg' => 'required']);

    DB::table('feedbacks')->insert([
        'account_id' => session('client_id'),
        'message' => $request->msg,
        'created_at' => now(),
        'updated_at' => now(),
    ]);

    return redirect('/client/feedback')->with('status', 'Votre message a été envoyé avec succès');
}
```

resources/views/Client/feedback.blade.php (extrait)

```
<form method="POST" action="{{ url('/client/savefeedback') }}">
        @csrf
        <div class="alert alert-success w-50 mx-auto">
          <h5>Enter your message</h5>
          <textarea class="form-control" name="msg" required placeholder="Write your message"></textarea>
          <button type="submit" name="send" class="btn btn-primary btn-block btn-sm my-1">Send</button>
        </div>
      </form>
```

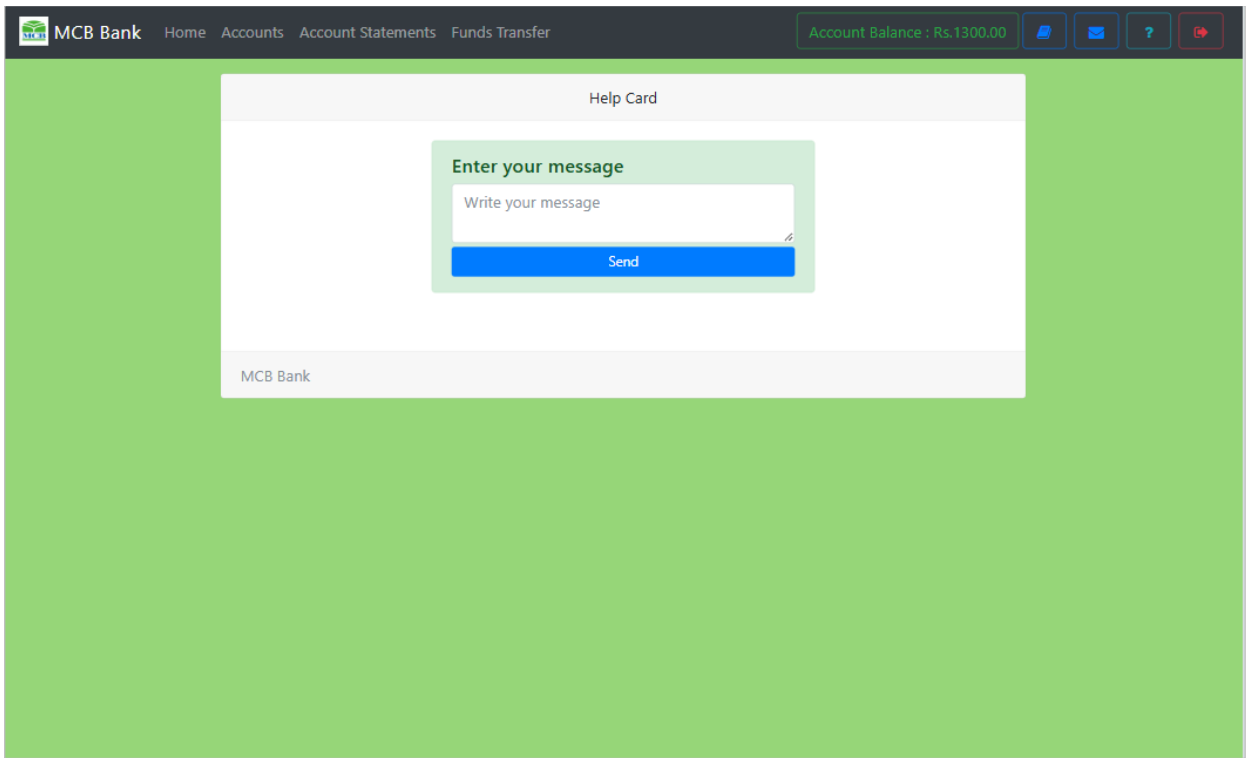


Figure 25 : *Icône « ? » : formulaire d'envoi d'un message à la banque*

5.6 Consultation des notices

ClientController

```
// Liste des notices reçues
public function notice()
{
    $notices = DB::table('notices')
        ->where('account_id', session('client_id'))
        ->orderBy('created_at', 'desc')
        ->get();
    return view('Client.notice', ['notices' => $notices]);
}
```

resources/views/Client/notice.blade.php (extrait)

```
@forelse ($notices as $notice)
        <div class="alert alert-success">Dear {{ $client->name }},<br>{{ $notice->notice }}
          <br><small>{{ $notice->created_at }}</small>
        </div>
      @empty
        <div class="alert alert-secondary">Aucune notification</div>
      @endforelse
```

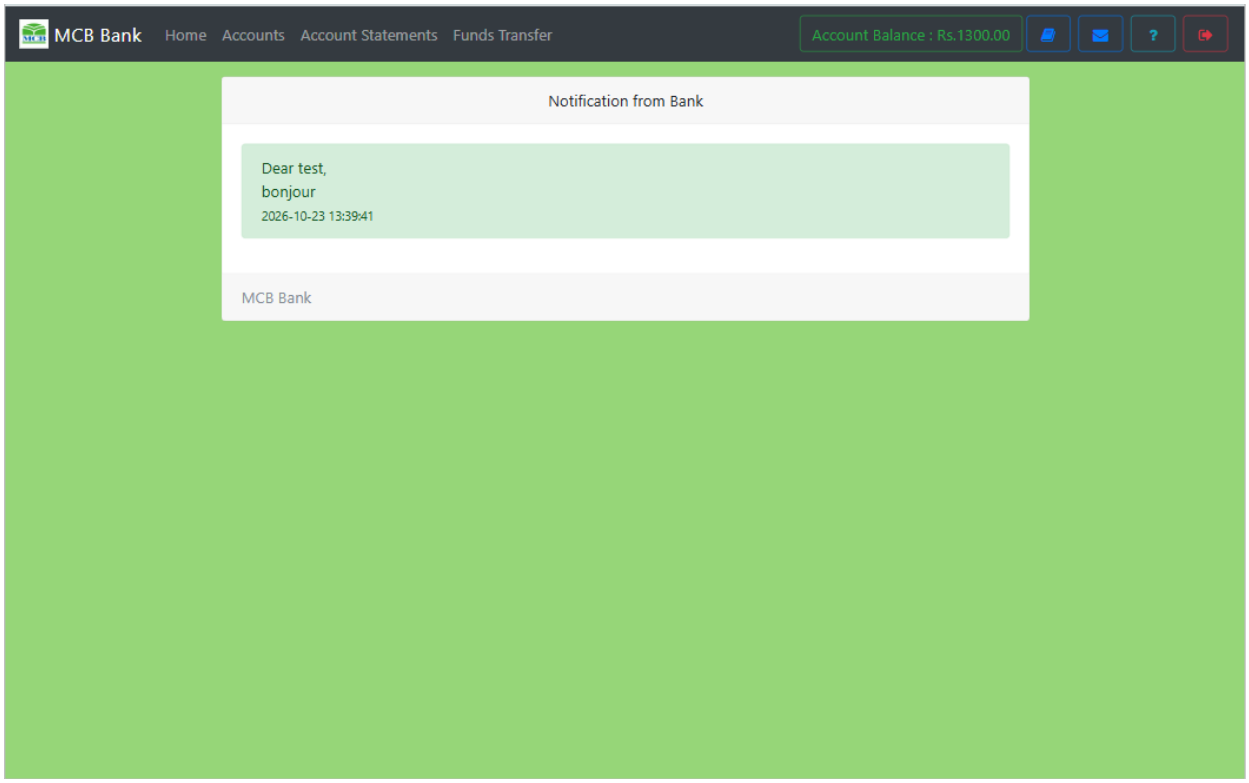


Figure 26 : *Icône enveloppe : notices envoyées par la banque*

6. Espace Caissier (/cashier)

Le caissier recherche un compte par son numéro puis effectue un dépôt ou un retrait.

6.1 Recherche d'un compte, dépôt et retrait

CashierController

```
// Page d'accueil caissier : rechercher un compte
public function home()
{
    return view('Admin.cindex');
}
```

```
// Afficher le compte trouvé avec le formulaire dépôt / retrait
public function search(Request $request)
{
    $request->validate(['otherNo' => 'required']);

    $account = DB::table('accounts')->where('accountNo', $request->otherNo)->first();
    if (!$account) {
        return redirect('/cashier/home')->with('error', 'Aucun compte ne correspond à ce numéro');
    }
    return view('Admin.cindex1', ['account' => $account]);
}
```

```
// Dépôt (bouton deposit) ou retrait (bouton withdraw)
public function operation(Request $request)
{
    $request->validate([
        'otherNo' => 'required|exists:accounts,accountNo',
        'amount' => 'required|numeric|min:1',
    ]);

    $account = DB::table('accounts')->where('accountNo', $request->otherNo)->first();

    if ($request->has('withdraw')) {
        // Retrait
        if ($account->balance < $request->amount) {
            return redirect('/cashier/home')->with('error', 'Solde insuffisant');
        }
        DB::table('accounts')->where('id', $account->id)->decrement('balance', $request->amount);
        DB::table('transactions')->insert([
            'account_id' => $account->id,
            'action' => 'withdraw',
            'debit' => $request->amount,
            'created_at' => now(),
            'updated_at' => now(),
        ]);
        $message = 'Retrait de Rs.'.$request->amount.' effectué';
    } else {
        // Dépôt
        DB::table('accounts')->where('id', $account->id)->increment('balance', $request->amount);
        DB::table('transactions')->insert([
            'account_id' => $account->id,
            'action' => 'deposit',
            'credit' => $request->amount,
            'created_at' => now(),
            'updated_at' => now(),
        ]);
        $message = 'Dépôt de Rs.'.$request->amount.' effectué';
    }

    return redirect('/cashier/home')->with('status', $message.' sur le compte '.$account->accountNo);
}
```

Dans la vue des opérations, chaque formulaire envoie le numéro de compte dans un champ caché :

resources/views/Admin/cindex1.blade.php (extrait)

```
{{-- Colonne droite : retrait et dépôt --}}
          <div class="col">
            Transaction Process.
            <form method="POST" action="{{ url('/cashier/operation') }}">
              @csrf
              <input type="hidden" name="otherNo" value="{{ $account->accountNo }}">
              <div class="form-group">
                <input type="number" class="form-control my-1" name="amount" placeholder="Write Amount for
withdraw" min="1" max="{{ $account->balance }}" required>
              </div>
              <button type="submit" name="withdraw" class="btn btn-primary btn-sm my-1">Withdraw</button>
            </form>
```

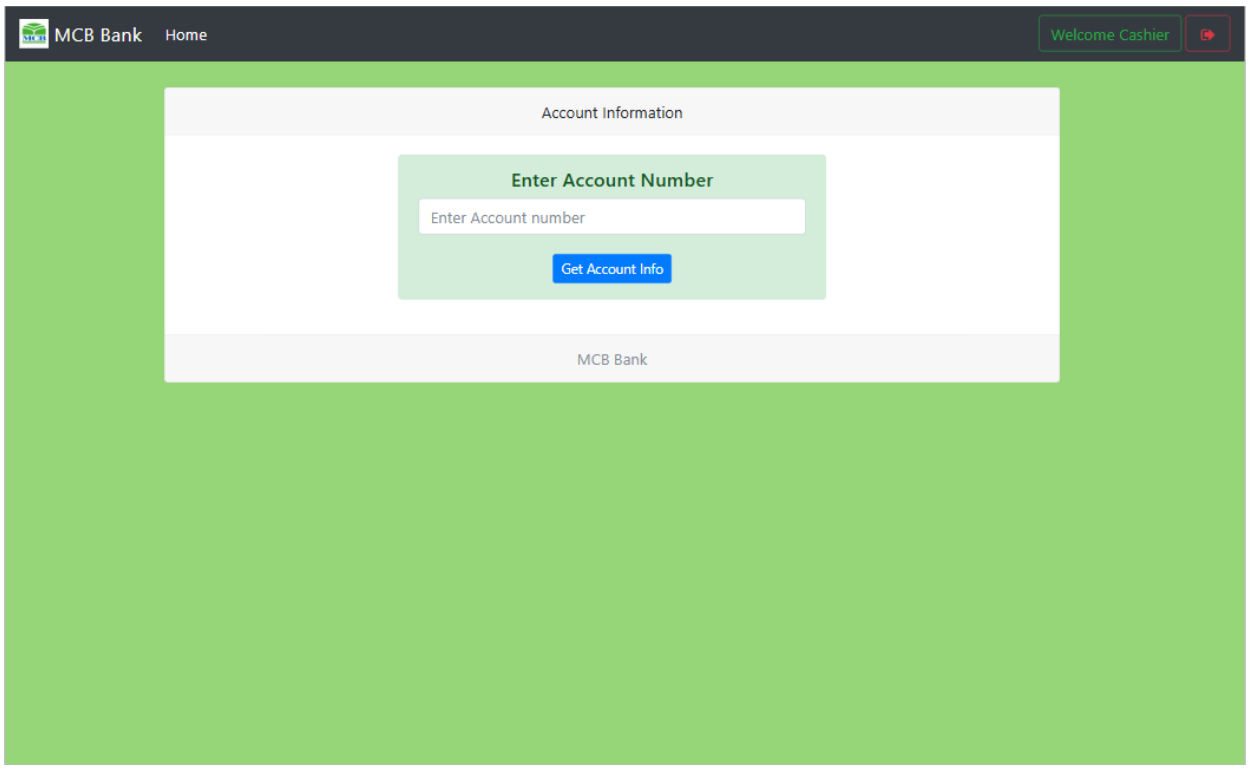


Figure 27 : *Connexion caissier : recherche d'un compte par son numéro*

7. Tests et bilan

Organisation finale du code, scénario de test complet et bilan du TP.

7.1 Organisation du code : commentaires et arborescence

Commentaires

Chaque vue commence par un commentaire Blade `{{-- ... --}}` qui indique la route de la page et son rôle. Ces commentaires n'apparaissent pas dans le code source envoyé au navigateur.

Exemple : resources/views/Admin/maccounts.blade.php

```
{{--
    GESTION DES CAISSIERS - CRUD  (route : /admin/cashiers)
    Read : liste des caissiers
    Create : modale "New Cashier Account"  -> POST /admin/savecashier
    Update : une modale "Edit" par caissier -> PUT /admin/updatecashier/{id}
    Delete : bouton Delete                  -> DELETE /admin/deletecashier/{id}
--}}
```

Arborescence de l'application

Sous Windows, la commande `tree` n'accepte qu'un seul dossier à la fois :

```
tree /F app\Http
tree /F routes
tree /F resources\views
```

Arborescence

```
app/Http
├── Kernel.php
├── Controllers
│   ├── AdminController.php
│   ├── AuthController.php
│   ├── CashierController.php
│   ├── ClientController.php
│   └── Controller.php
└── Middleware
    └── CheckRole.php   (+ middlewares par défaut de Laravel)
routes
└── web.php
resources/views
├── login.blade.php
├── messages.blade.php
├── Admin
│   ├── cindex.blade.php      (caissier : recherche)
│   ├── cindex1.blade.php     (caissier : dépôt / retrait)
│   ├── home.blade.php        (liste des comptes)
│   ├── maccounts.blade.php   (CRUD caissier)
│   ├── maddnew.blade.php     (création de compte)
│   ├── mfeedback.blade.php   (feedbacks)
│   ├── mnotice.blade.php     (envoi de notice)
│   └── mshow.blade.php       (détails client)
├── AdminLayout
│   ├── header.blade.php      (navbar manager)
│   ├── master.blade.php
│   └── sidebar.blade.php     (navbar caissier)
├── Client
│   ├── accounts.blade.php    (détails du compte)
│   ├── feedback.blade.php
│   ├── home.blade.php
│   ├── notice.blade.php
│   ├── statements.blade.php  (relevé)
│   ├── transfe.blade.php     (transfert, étape 2)
│   └── transfer.blade.php    (transfert, étape 1)
└── ClientLayout
    ├── header.blade.php      (navbar client)
    └── master.blade.php
```

```
public/frontend   (css, js, images, font-awesome du template)
```

```
Terminal — tree /F app\Http

PS C:\wamp64\www\MCBBank> tree /F app\Http
C:\WAMP64\WWW\MCBBANK\APP\HTTP
|   Kernel.php
|
+---Controllers
|       AdminController.php
|       AuthController.php
|       CashierController.php
|       ClientController.php
|       Controller.php
|
\---Middleware
        CheckRole.php
        (+ middlewares par défaut de Laravel)
```

Figure 28 : *Terminal : tree /F app\Http*

```
Terminal — tree /F resources\views

PS C:\wamp64\www\MCBBank> tree /F resources\views
C:\WAMP64\WWW\MCBBANK\RESOURCES\VIEWS
|   login.blade.php
|   messages.blade.php
+---Admin
|       cindex.blade.php      cindex1.blade.php    home.blade.php
|       maccounts.blade.php   maddnew.blade.php    mfeedback.blade.php
|       mnotice.blade.php     mshow.blade.php
+---AdminLayout
|       header.blade.php      master.blade.php     sidebar.blade.php
+---Client
|       accounts.blade.php    feedback.blade.php   home.blade.php
|       notice.blade.php      statements.blade.php
|       transfe.blade.php     transfer.blade.php
\---ClientLayout
        header.blade.php      master.blade.php
```

Figure 29 : *Terminal : tree /F resources\views (ou explorateur de VS Code)*

7.2 Scénario de test

1. **Manager** : connexion → création de 2 comptes clients → View → création d'un caissier → Edit → Send Notice.
2. **Client 1** : lecture de la notice → Accounts → transfert vers le client 2 → Statements → Feedback.
3. **Caissier** : dépôt et retrait sur le client 2.
4. **Client 2** : Statements (transfert reçu, dépôt, retrait).
5. **Manager** : Feedback → Delete → suppression d'un compte client → suppression du caissier.

7.3 Récapitulatif des fonctionnalités de l'énoncé

Fonctionnalité	Route(s)	Réalisé
Caissier (CRUD)	`/admin/cashiers`, `savecashier`, `updatecashier/{id}`, `deletecashier/{id}`	Oui
Création de compte bancaire	`/admin/addnew`, `/admin/saveaccount`	Oui
Afficher les détails du client	`/admin/show/{id}`	Oui
Supprimer le compte client	`/admin/deleteaccount/{id}`	Oui
Note de feedback de la banque	`/client/feedback`, `/client/savefeedback`	Oui
Connexion du compte	`/login`, `/logout`	Oui
Suppression de feedback	`/admin/feedbacks`, `/admin/deletefeedback/{id}`	Oui
Les détails de compte bancaire	`/client/account`	Oui
Transfert de fonds	`/client/transfer`, `transferinfo`, `savetransfer`	Oui
Relevé bancaire	`/client/statements`	Oui
Bonus : notices, dépôt / retrait	`/admin/notice/{id}`, `/client/notice`, `/cashier/...`	Oui

7.4 Query Builder et Eloquent

Ce TP utilise le Query Builder. Avec le Query Builder, il faut remplir soi-même `created_at` et `updated_at` avec `now()`, alors qu'Eloquent le fait automatiquement.

Action	Query Builder (ce TP)	Eloquent
Tout lire	`DB::table('accounts')->get()`	`Account::all()`
Un seul	`DB::table('accounts')->where('id',$id)->first()`	`Account::find($id)`
Insérer	`DB::table('accounts')->insert([...])`	`$a = new Account(); ... $a->save();`
Modifier	`DB::table('staff')->where('id',$id)->update([...])`	`$c = Staff::find($id); ... $c->save();`
Supprimer	`DB::table('accounts')->where('id',$id)->delete()`	`Account::find($id)->delete()`
Pagination	`DB::table('transactions')->paginate(10)`	`Transaction::paginate(10)`

7.5 Conclusion

Ce TP m'a permis de transformer un template HTML statique en application Laravel dynamique : factorisation du code commun avec le Blade templating (`@extends`, `@section`, `@yield`, `@include`), protection des pages par rôle avec un middleware, manipulation de la base de données avec le Query Builder (lecture, insertion, modification, suppression, jointure, pagination, transaction), validation des formulaires et envoi de fichiers.

Difficulté rencontrée : l'erreur « La clé est trop longue » due au moteur MyISAM de WAMP, résolue en passant au moteur InnoDB.